

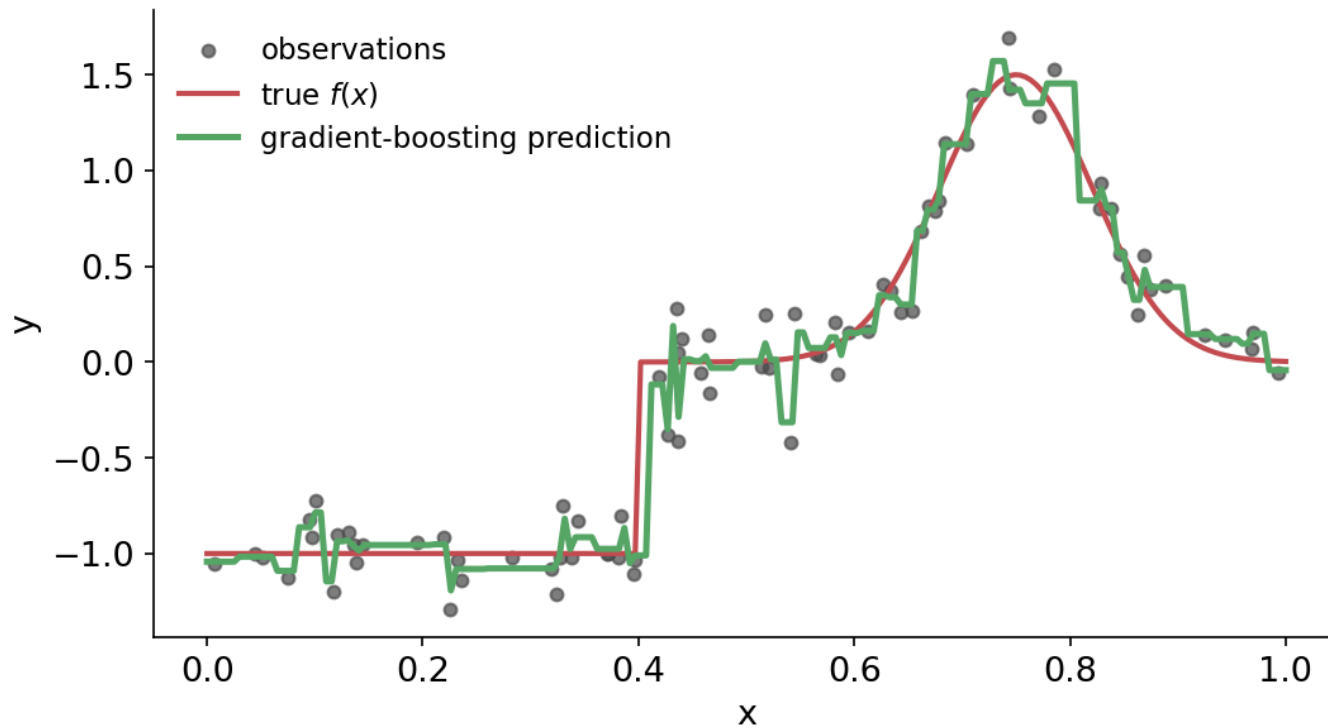
How BART works

Bayesian Additive Regression Trees, from scratch

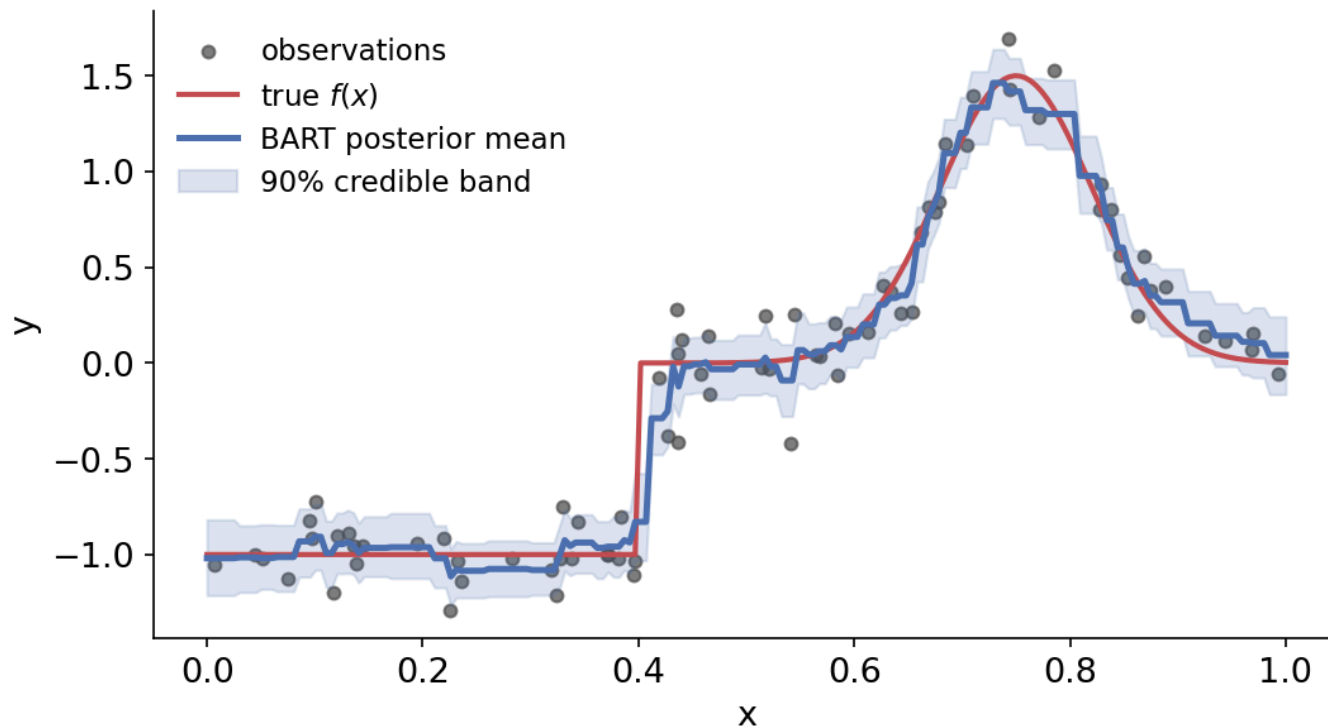
Chris Fonnesbeck · PyMC Labs

Start with a familiar model

Gradient boosting



Same data, with BART



So what is BART?

STRUCTURE

Like gradient boosting — a prediction is a sum of many small trees.

FITTING

Not stagewise boosting — posterior backfitting, not greedy residual chasing.

OUTPUT

Bayesian — posterior draws over functions give credible bands.

Bayesian inference, briefly

Bayes' rule

$$\underbrace{P(\theta \mid D)}_{\text{posterior}} = \frac{\overbrace{P(D \mid \theta)}^{\text{likelihood}} \cdot \overbrace{P(\theta)}^{\text{prior}}}{\underbrace{P(D)}_{\text{evidence}}}$$

The Gaussian model

Priors

Mean:

$$\mu \sim \mathcal{N}(\mu_0, \tau^2)$$

Noise:

$$\sigma^2 \sim \text{Inv-}\chi^2(\nu, \lambda)$$

Likelihood

Given the mean and noise variance:

$$y_i \mid \mu, \sigma^2 \sim \mathcal{N}(\mu, \sigma^2)$$

Bayes combines these into conditional updates for μ and σ^2 .

Conditional update for the mean

Given the current σ^2 , Normal prior + Normal likelihood gives:

$$\mu \mid y \sim \mathcal{N} \left(\underbrace{\frac{\tau^{-2} \mu_0 + \sigma^{-2} n \bar{y}}{\tau^{-2} + \sigma^{-2} n}}_{\text{precision-weighted average}}, \underbrace{(\tau^{-2} + \sigma^{-2} n)^{-1}}_{\text{combined precision}} \right)$$

MCMC, briefly

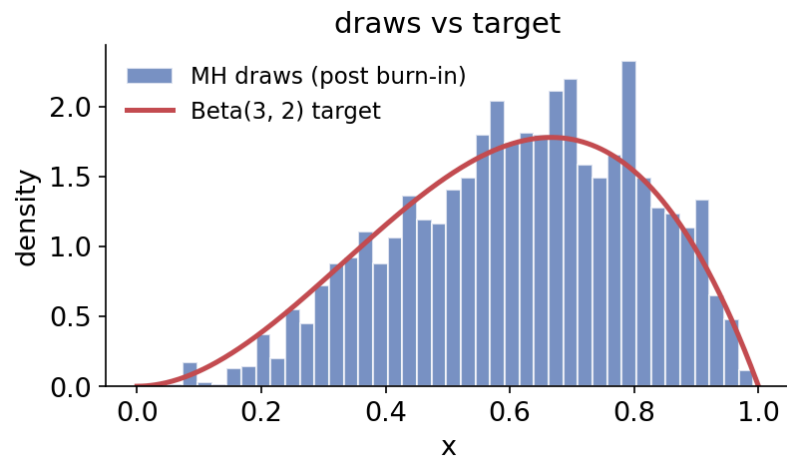
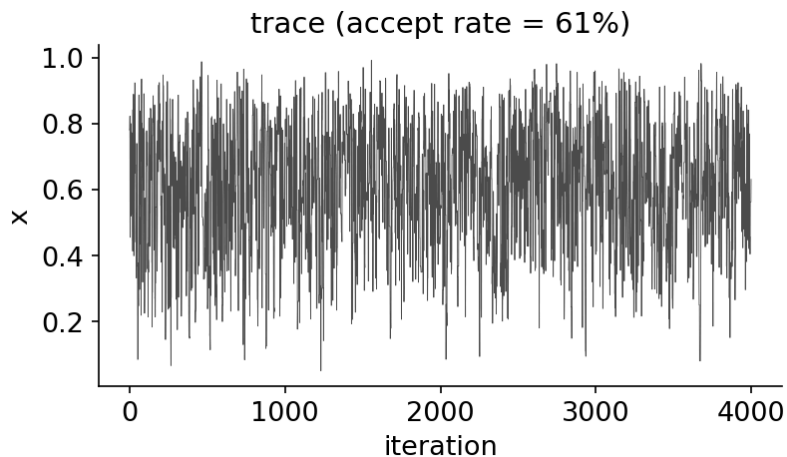
Metropolis– Hastings

Propose θ' , accept with probability

$$\alpha = \min \left(1, \underbrace{\frac{P(\theta' | D)}{P(\theta | D)}}_{\text{posterior ratio}} \cdot \underbrace{\frac{q(\theta | \theta')}{q(\theta' | \theta)}}_{\text{proposal correction}} \right)$$

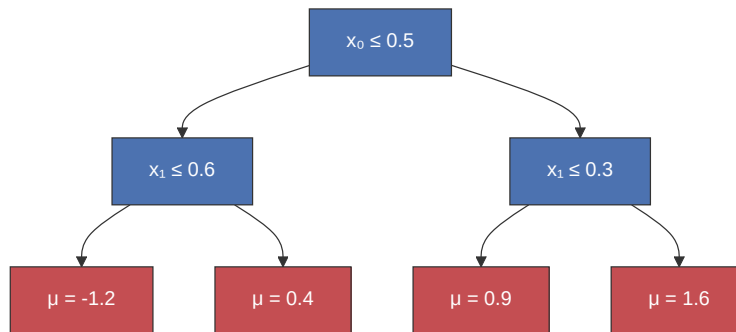
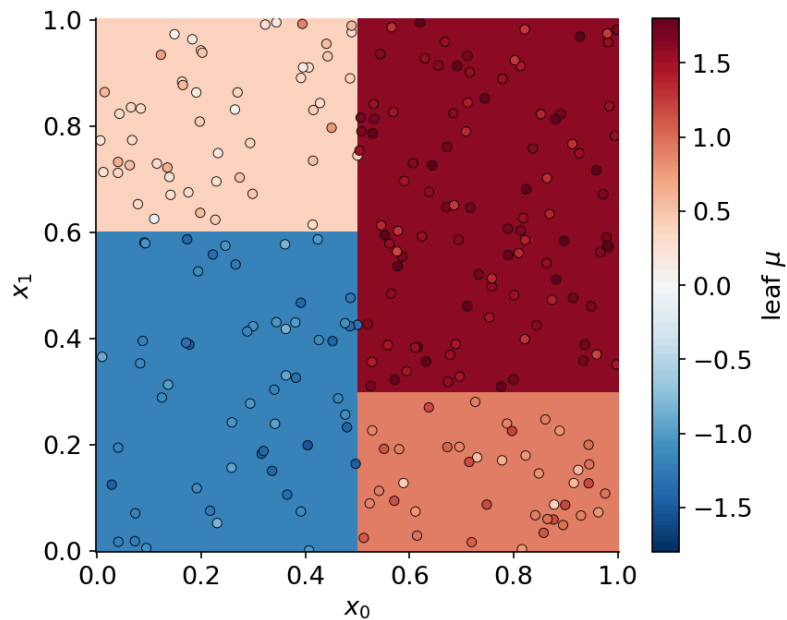
Metropolis–Hastings in seven lines

```
# random-walk Metropolis on a Beta(3, 2) target
for t in range(n_draws):
    x_new = x + rng.normal(scale=step)
    lp_new = log_target(x_new)
    if np.log(rng.uniform()) < lp_new - lp:    # accept?
        x, lp = x_new, lp_new
    draws[t] = x
```

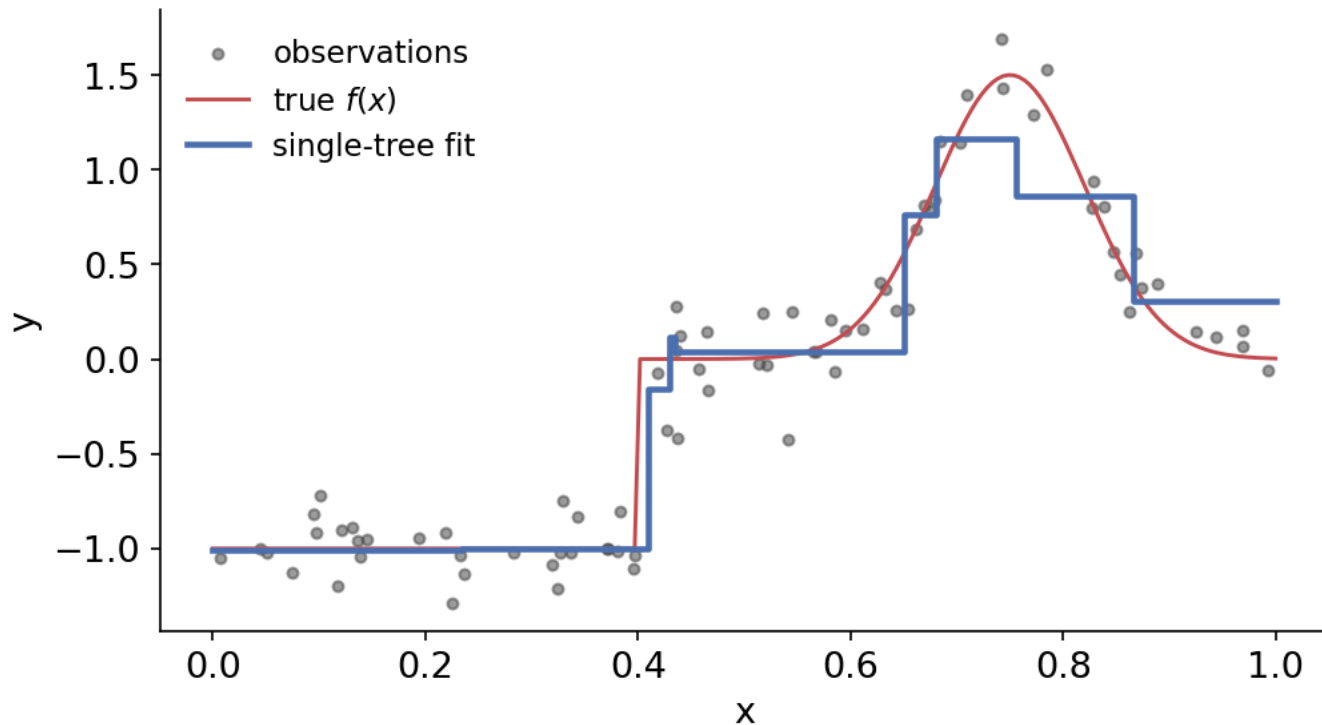


A single regression tree

A tree partitions the input space

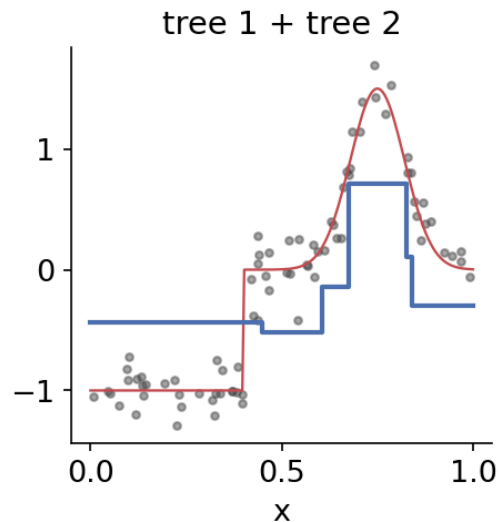
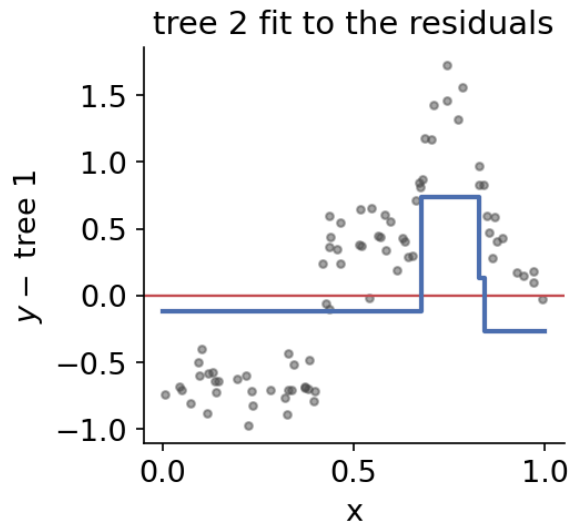
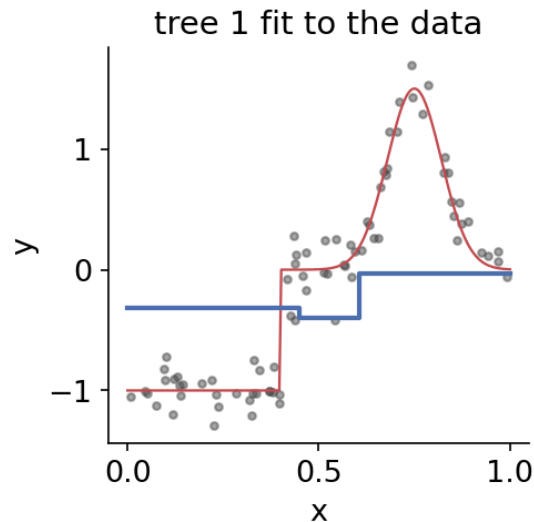


A tree predicts the leaf mean



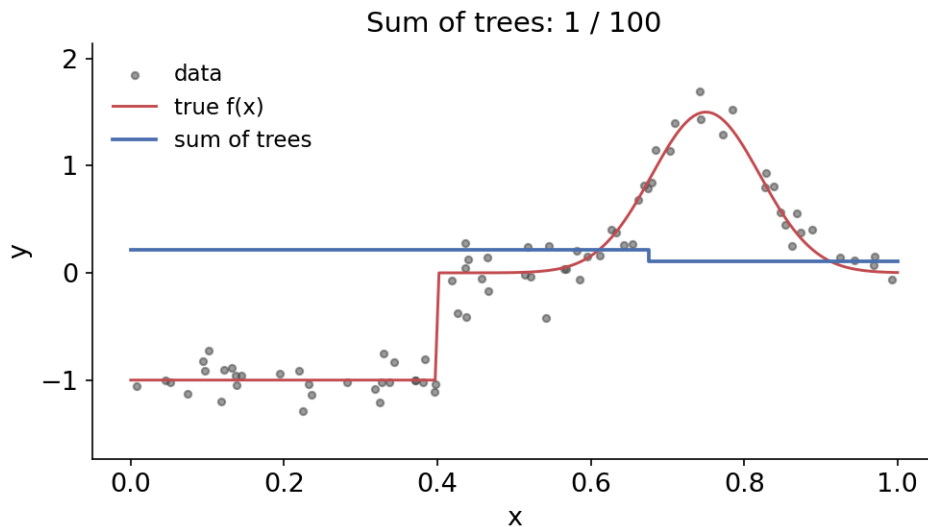
Sum of trees

Fit the next tree to the residuals



BART is additive

$$f(x) = \sum_{j=1}^m g(x; T_j, M_j)$$



Priors instead of cross-validation

TREE SHAPE

Boosting caps depth by CV — BART puts a **prior on depth**.

LEAF VALUES

Boosting tunes a learning rate — BART **shrinks leaves** toward zero.

NOISE

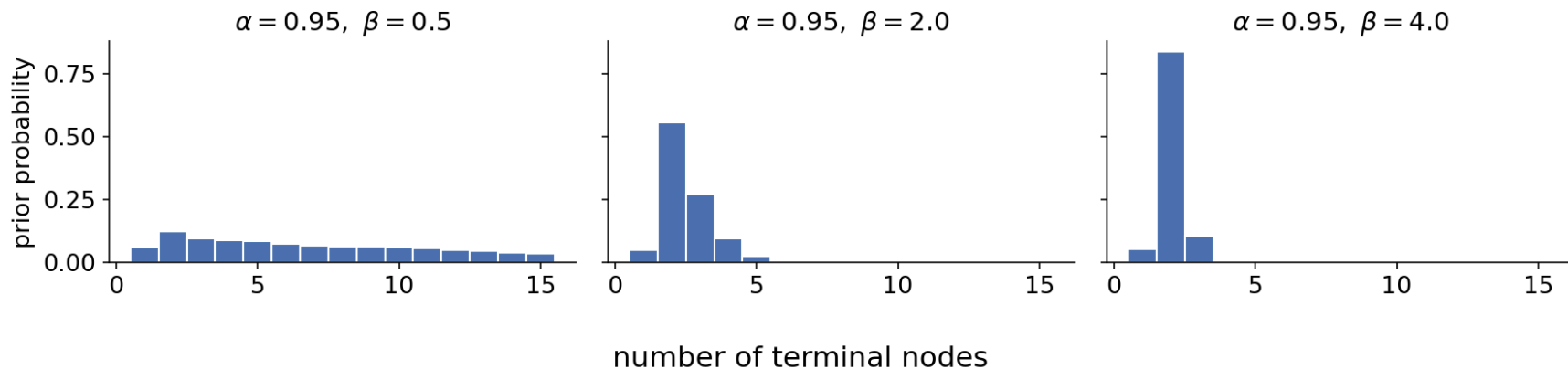
BART anchors σ^2 **just below the OLS residual variance**.

Prior on tree shape

Depth is penalised exponentially

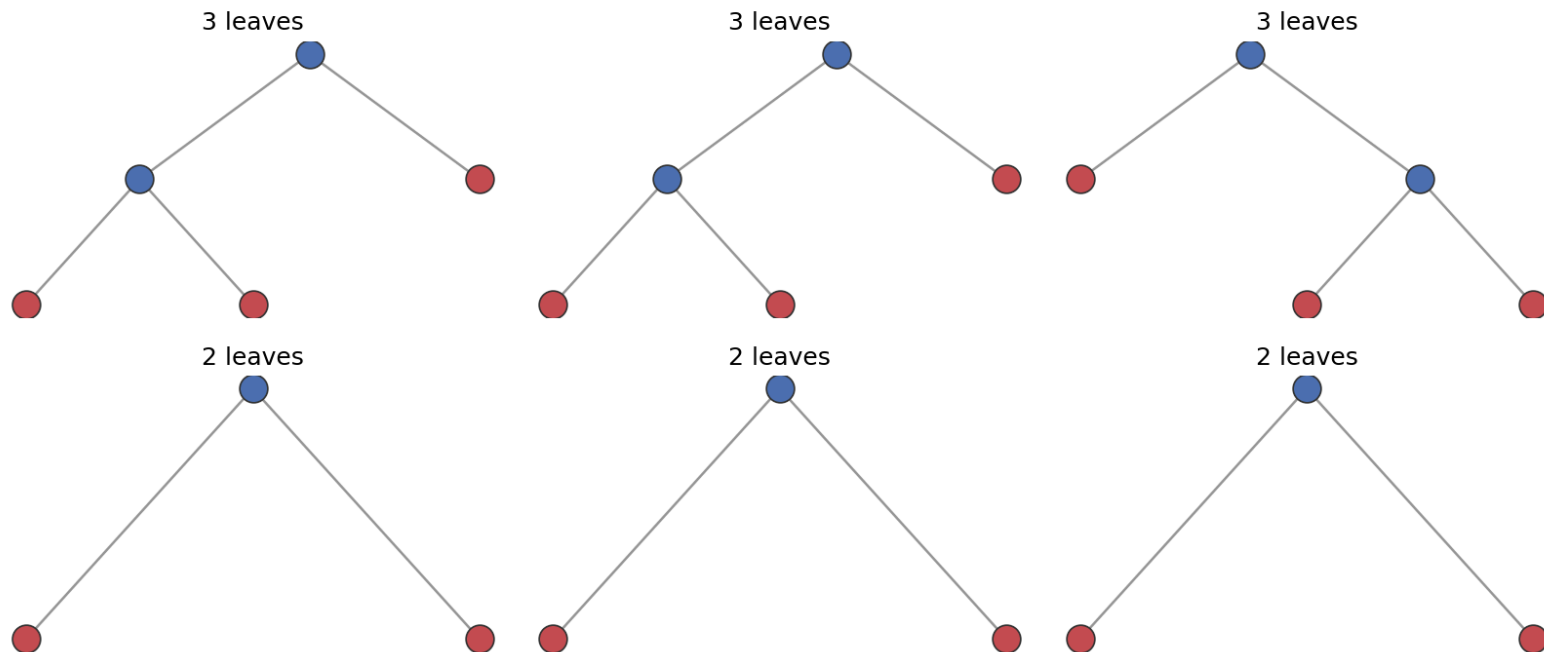
Each split decision at depth d is Bernoulli with $P(\text{split}) = \alpha (1 + d)^{-\beta}$ (Chipman et al. 1998)

Higher β punishes depth harder $\rightarrow$ smaller trees



Trees drawn from the prior

Six trees drawn from the prior ($\alpha = 0.95$, $\beta = 2.0$)



Leaf values are shrunk averages

LEAF VALUE

A leaf stores the average leftover signal for rows that land there.

SHRINKAGE

BART pulls that average toward zero before adding it to the forest.

EFFECT

Each tree contributes a small correction; the forest carries the pattern.

Metropolis–Hastings on tree structure

Grow and prune

Grow

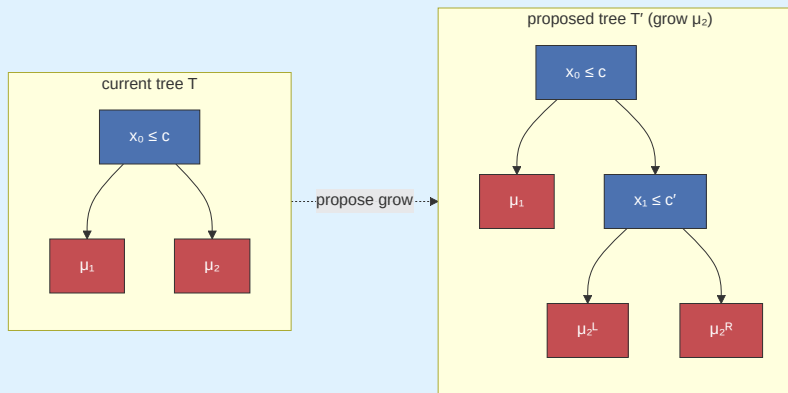
Split one existing leaf into two smaller leaves.

Prune

Collapse a tiny branch back into one leaf.

Accept

Keep moves that improve residual fit enough to justify a larger tree.



The acceptance ratio

$$\log A = \underbrace{\log \frac{p(r \mid T')}{p(r \mid T)}}_{\text{likelihood}} + \underbrace{\log \frac{\pi(T')}{\pi(T)}}_{\text{prior shape}} + \underbrace{\log \frac{q(T \mid T')}{q(T' \mid T)}}_{\text{move}}$$

Accept T' with probability $\min(1, e^{\log A})$. **Prune is the time-reversal of grow.**

The σ update

σ tells you what BART has not explained

$$y = \underbrace{f(x)}_{\text{signal}} + \underbrace{\epsilon}_{\text{leftover noise}}$$

After the forest captures structure, σ is estimated from the remaining residual scatter.

INTERVALS

Larger σ means wider predictive intervals.

CHECK

Inflated σ means the forest is leaving pattern behind.

TREE COUNT

Too few trees often show up as too much leftover noise.

Putting it together

Blocked Gibbs

one BART sweep

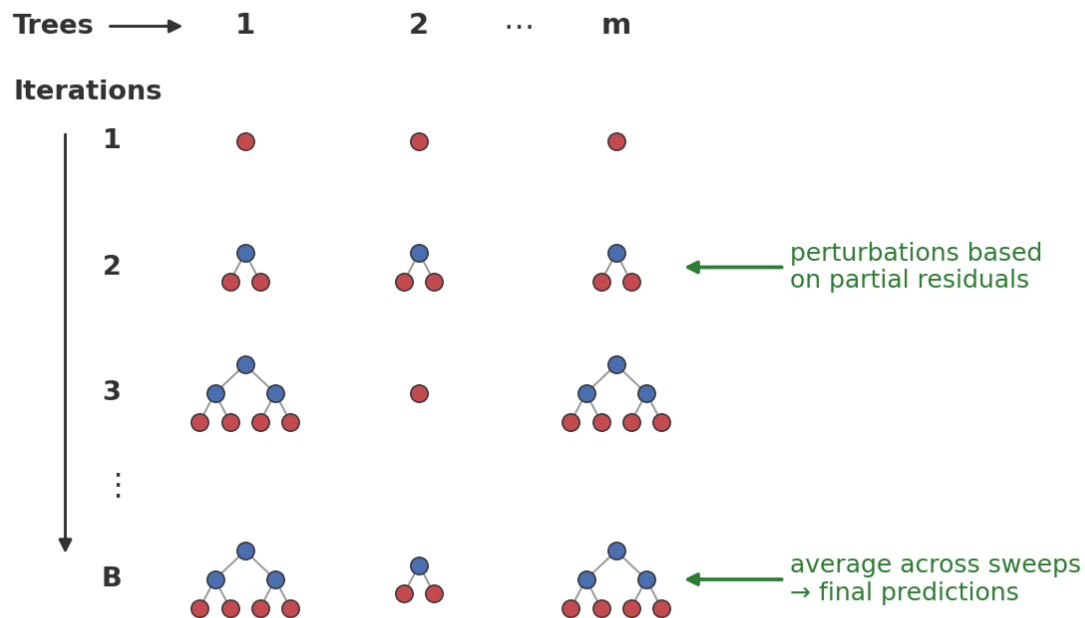
1. **Backfit** — isolate tree j 's residual signal

$$R_j = y - \sum_{i \neq j} g(x; T_i, M_i)$$

2. **Structure** — MH grow/prune move for T_j
3. **Leaf values** — conjugate draw of M_j
4. **Noise** — inverse- χ^2 draw of σ^2

A sweep is **Gibbs sampling** over these four blocks.

The BART algorithm



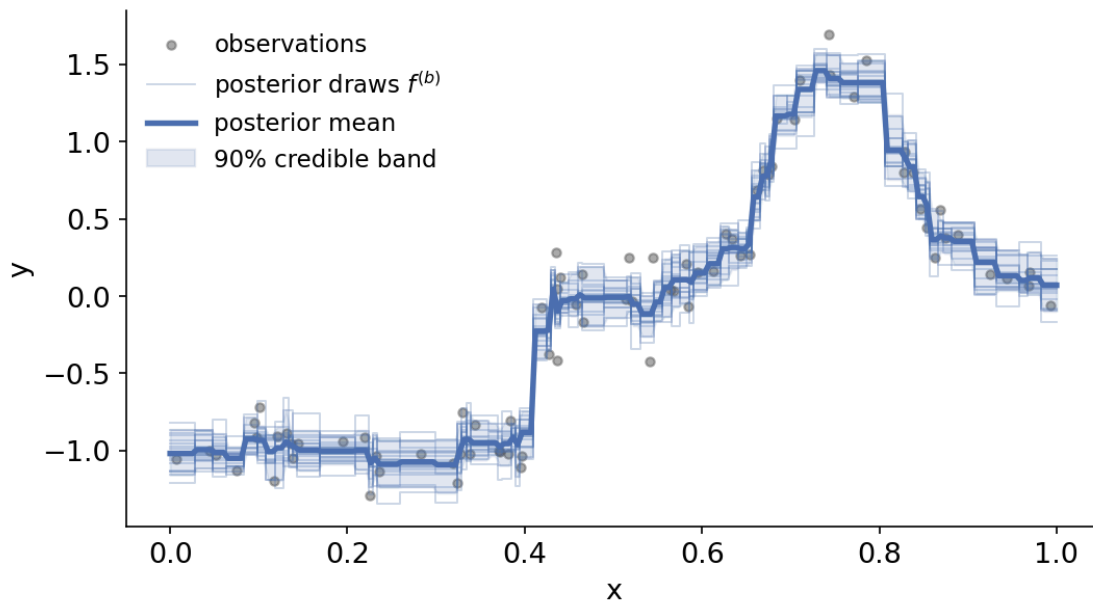
The sampler loop

```
for draw in range(n_draws):  
    for tree in forest:  
        residual = y - prediction_from_other_trees()  
        tree = update_structure(tree, residual)  
        tree = update_leaf_values(tree, residual)  
  
    sigma = update_leftover_noise(y - forest.predict(X))  
    save(forest.predict(X), sigma)
```

Each saved draw is one plausible function; the collection of draws becomes the prediction interval.

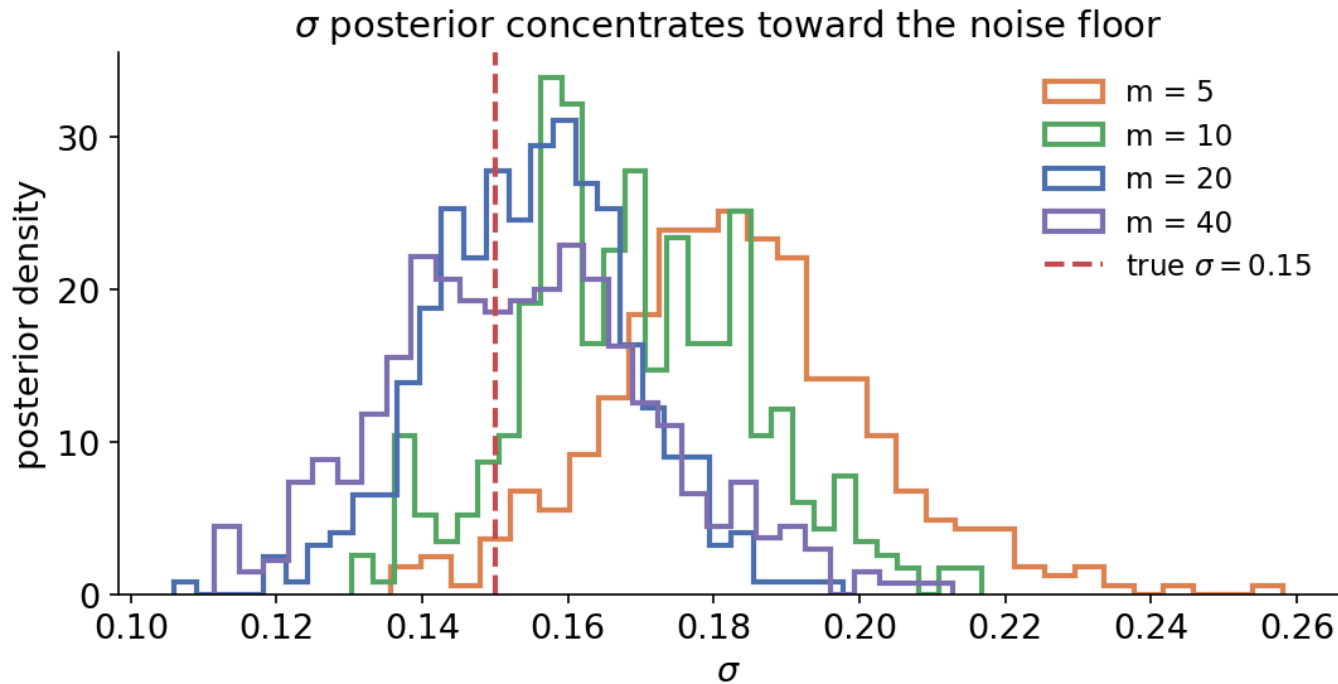
What BART delivers

$$\hat{f}(x) = \frac{1}{B - L} \sum_{b=L+1}^B \underbrace{f^{(b)}(x)}_{\text{one draw per sweep}}$$

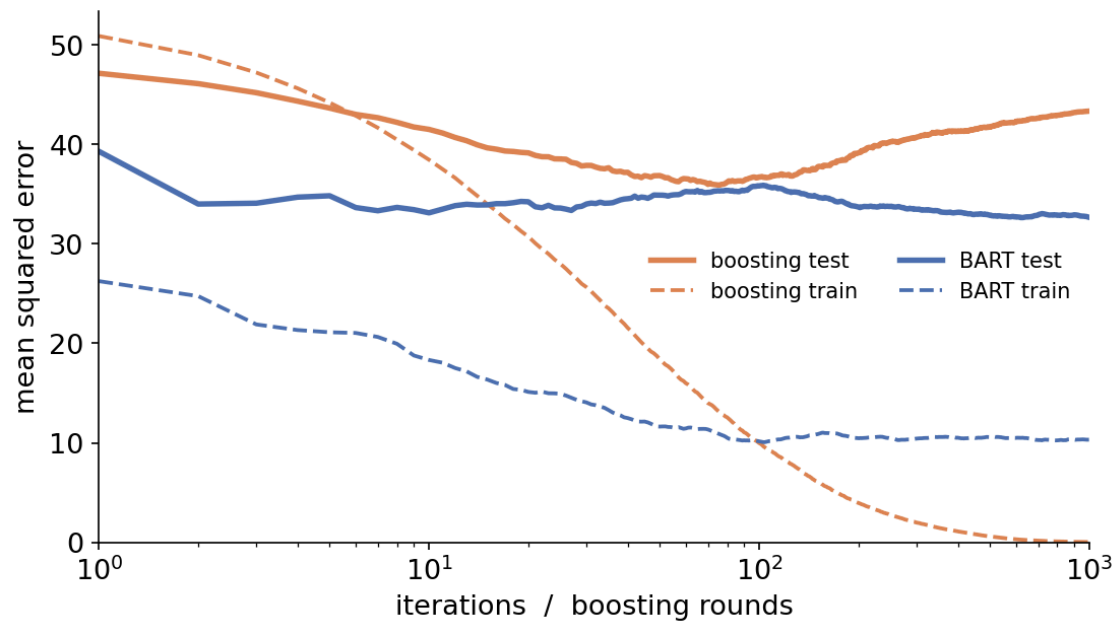


Choosing m

Too few trees inflate $\hat{\sigma}$



Sampling longer is not adding complexity



Let's run some code!